

R2D2 Prior

General Principles

Rather than setting arbitrary, independent variances for individual regression coefficients — which can be difficult to intuit — the **R2D2** (*R*-squared induced Dirichlet Decomposition) prior places a prior directly on a macro-level, easily interpretable metric: the model’s overall goodness-of-fit, R^2 . This prior was introduced in (gelman2022r2d2?).

Think of it as a **variance budget**. A single prior on R^2 opens a pot of “explained variance”; a Dirichlet simplex then divides that pot among the predictors; each predictor’s slice becomes the prior variance of its coefficient. The generative process has five steps (Figure ??):

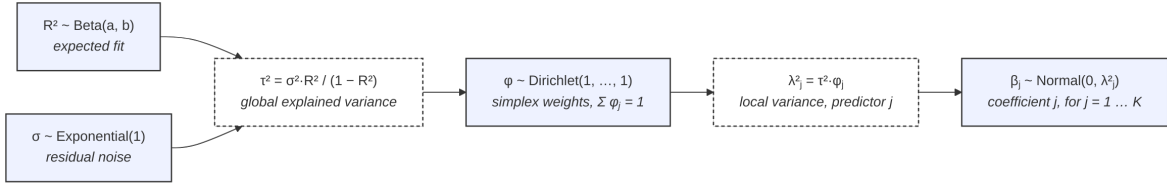


Figure 1: The R2D2 prior as a variance budget: a Beta prior on R^2 — together with the residual scale σ — opens a global explained variance τ^2 ; a flat Dirichlet splits it into per-predictor weights ϕ_j ; and each slice $\lambda_j^2 = \tau^2 \phi_j$ becomes the prior variance of coefficient β_j . Dashed nodes are deterministic transforms.

1. **Set the expectation.** $R^2 \sim \text{Beta}(a, b)$ — how much of y ’s variance do you expect this model to explain? The conservative default is $\text{Beta}(1/3, 3)$ (prior mean $R^2 = 0.1$).
2. **Open the budget.** $\tau^2 = \frac{\sigma^2 R^2}{1 - R^2}$ maps the R^2 belief onto a total signal variance, scaled by the residual variance σ^2 . This τ^2 is the signal-to-noise budget available to the coefficients (see *Definition of R^2* for the exact quantity R^2 refers to).
3. **Split it.** $\phi \sim \text{Dirichlet}(a_0, \dots, a_0)$ is a vector of K non-negative weights that sum to one. The concentration a_0 controls how sparse the split is: $a_0 = 1$ (the usual default) gives every predictor the same prior share — ridge-like — while $a_0 < 1$ favours allocating the

budget to a few predictors, which makes the induced coefficient priors heavier-tailed and more horseshoe-like.

4. **Hand out slices.** $\lambda_j^2 = \tau^2 \phi_j$ is predictor j 's share of the budget.

5. **Draw the coefficient.** $\beta_j \sim \text{Normal}(0, \lambda_j^2)$.

i Note

The budget metaphor. Because the Dirichlet weights sum to one, the predictors *compete* for a fixed pot: a predictor that claims a large share leaves less for the rest, so unsupported predictors are handed a tiny ϕ_j and their coefficients collapse toward zero — adaptive sparsity without an explicit spike-at-zero. This reading only holds if the predictors are **standardized**: $\lambda_j^2 = \tau^2 \phi_j$ distributes *variance*, so predictors on incomparable scales would confound “share of explained variance” with unit of measurement.

Considerations

i Note

- Instead of declaring independent prior distributions for each coefficient variance, the R2D2 prior encodes a single macro-level belief: the fraction of the outcome variance the model is expected to explain. Eliciting expert knowledge on an expected R^2 is much easier than guessing the variance of an abstract regression weight.
- Because the Dirichlet weights ϕ_j must sum to one, the predictors are forced to “compete” for a share of the total variance. If one predictor takes a larger share, the others are inherently penalized. This *adaptive sparsity* naturally shrinks irrelevant variables towards zero and helps prevent overfitting.
- The induced coefficient priors concentrate probability mass around zero while maintaining heavy tails: noise is strongly regularized, yet genuinely large coefficients are estimated without bias.
- The framework extends to **varying intercepts** and other **hierarchical** models: the simplex is expanded so that the variance components of the grouping factors receive their own slice of the global variance τ^2 , forcing fixed covariates and group-level effects to compete for the total explained variance.
- In mixed-effects models, the Beta prior must target the *conditional* R^2 (variance explained by fixed plus varying effects), not the *marginal* R^2 (fixed effects alone), so that the full signal-to-noise ratio is captured before the Dirichlet weights decompose it down the hierarchy.

Example

Below is an example code snippet demonstrating an R^2D^2 prior on the regression coefficients using the Bayesian Inference (**BI**) package. A Beta prior on the expected R^2 is transformed into a global variance, which is then decomposed through a Dirichlet simplex into local variances for each of the two predictors (weight and age). This example is based on (gelman2022r2d2?).

Python

```
from BayesForge import bf
import jax.numpy as jnp

# Setup device-----
m = bf(platform="cpu")

# Import Data & Data Manipulation -----
# Import
data_path = m.load.howell1(only_path=True)
m.data(data_path, sep=";")
m.df = m.df[m.df.age > 18] # Subset data to adults
m.scale(["weight", "age"]) # Normalize

# Define model -----
def model(height, weight, age):
    # R2D2 prior: Beta prior on the expected R^2 of the model
    r2 = m.dist.beta(1 / 3, 3, name="r2")
    sigma = m.dist.exponential(1, name="s")
    # Global (total explained) variance
    tau2 = sigma**2 * r2 / (1 - r2)
    # Dirichlet weights over the K = 2 predictors
    phi = m.dist.dirichlet(jnp.ones(2), name="phi")
    # Local coefficient variances
    lam2 = tau2 * phi
    # Parameter prior distributions
    alpha = m.dist.normal(0, 0.5, name="alpha")
    beta1 = m.dist.normal(0, jnp.sqrt(lam2[0]), name="beta1")
    beta2 = m.dist.normal(0, jnp.sqrt(lam2[1]), name="beta2")
    # Likelihood
    m.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs=height)

# Run MCMC -----
```

```

m.fit(model, progress_bar=False) # Optimize model parameters through MCMC sampling

# Summary -----
m.summary() # Get posterior distributions

```

R

```

library(BayesianInference)
m=importBI(platform='cpu')

# Import Data & Data Manipulation -----
m$data(m$load$howell1(only_path = T), sep=';') # Import
m$df = m$df[m$df$age > 18,] # Subset data to adults
m$scale(list('weight', 'age')) # Normalize
m$data_to_model(list('weight', 'height', 'age')) # Send to model (convert to jax array)

# Define model -----
model <- function(height, weight, age){
  # R2D2 prior: Beta prior on the expected R^2 of the model
  r2 = bi.dist.beta(1 / 3, 3, name = 'r2')
  sigma = bi.dist.exponential(1, name = 's')
  # Global (total explained) variance
  tau2 = sigma^2 * r2 / (1 - r2)
  # Dirichlet weights over the K = 2 predictors
  phi = bi.dist.dirichlet(c(1, 1), name = 'phi')
  # Local coefficient variances
  lam2 = tau2 * phi
  # Parameter prior distributions
  alpha = bi.dist.normal(0, 0.5, name = 'a')
  beta1 = bi.dist.normal(0, sqrt(lam2[0]), name = 'b1')
  beta2 = bi.dist.normal(0, sqrt(lam2[1]), name = 'b2')
  # Likelihood
  bi.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs=height)
}

# Run MCMC -----
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary -----
m$summary() # Get posterior distributions

```

Julia

```
using BayesianInference

# Setup device-----
m = importBI(platform="cpu")

# Import Data & Data Manipulation -----
# Import
data_path = m.load.howell1(only_path = true)
m.data(data_path, sep=';')
m.df = m.df[m.df.age > 18] # Subset data to adults
m.scale(["weight", "age"]) # Normalize

# Define model -----
@BI function model(height, weight, age)
    # R2D2 prior: Beta prior on the expected R^2 of the model
    r2 = m.dist.beta(1 / 3, 3, name = "r2")
    sigma = m.dist.exponential(1, name = "s")
    # Global (total explained) variance
    tau2 = sigma^2 * r2 / (1 - r2)
    # Dirichlet weights over the K = 2 predictors
    phi = m.dist.dirichlet([1.0, 1.0], name = "phi")
    # Local coefficient variances
    lam2 = tau2 .* phi
    # Parameter prior distributions
    alpha = m.dist.normal(0, 0.5, name = "alpha")
    beta1 = m.dist.normal(0, sqrt(lam2[1]), name = "beta1")
    beta2 = m.dist.normal(0, sqrt(lam2[2]), name = "beta2")
    # Likelihood
    m.dist.normal(alpha + beta1 * weight + beta2 * age, sigma, obs = height)
end

# Run mcmc -----
m.fit(model) # Optimize model parameters through MCMC sampling

# Summary -----
m.summary() # Get posterior distributions
```

Example: Varying Effects

The Dirichlet simplex can be expanded to allocate variance to group-level effects alongside fixed predictors. Below, two components share the global variance τ^2 : one for a continuous predictor x and one for village-level varying intercepts. Because they compete for the same budget, a village effect that explains little is automatically shrunk without any separate prior on σ_{village} .

Python

```
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

m = bf(platform="cpu")

# Simulated data: 200 individuals in 5 villages
np.random.seed(0)
N, N_villages = 200, 5
village_id = np.random.randint(0, N_villages, N).astype("int32")
x          = np.random.normal(0, 1, N).astype("float32")
u_true     = np.random.normal(0, 0.6, N_villages)
y          = 1.2 + 0.8 * x + u_true[village_id] + np.random.normal(0, 0.8, N)

m.data_on_model = {
    "y":          jnp.array(y.astype("float32")),
    "x":          jnp.array(x),
    "village_id": jnp.array(village_id, dtype=jnp.int32),
    "N_villages": N_villages,
}

def model(y, x, village_id, N_villages):
    sigma = m.dist.exponential(1, name="sigma")

    # R2D2: 2-component simplex
    #   phi[0] → fixed predictor x
    #   phi[1] → village-level varying intercept
    r2     = m.dist.beta(1/3, 3, name="r2")
    tau2   = sigma**2 * r2 / (1 - r2)
    phi    = m.dist.dirichlet(jnp.ones(2), name="phi")
    lam2   = tau2 * phi
```

```

alpha      = m.dist.normal(0, 0.5, name="alpha")
beta       = m.dist.normal(0, jnp.sqrt(lam2[0]), name="beta")
u_village = m.dist.normal(0, jnp.sqrt(lam2[1]),
                          shape=(N_villages,), name="u_village")

mu = alpha + beta * x + u_village[village_id]
m.dist.normal(mu, sigma, obs=y, name="y_obs")

m.fit(model, progress_bar=False)
m.summary()

```

/home/sosa/work/3.12venv/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please use `from tqdm.auto import tqdm` in Python scripts or `from autonotebook import tqdm as notebook\_tqdm` in Jupyter notebooks.

An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Falls back to using CPU.

bf v 0.0.58 package loaded
jax.local_device_count 32

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd	
alpha		0.99	0.30	0.45	1.41	836.05	770.31	1.0	0.01	0.01
beta		0.73	0.06	0.64	0.83	2717.15	2350.94	1.0	0.00	0.00
phi[0]		0.47	0.19	0.18	0.78	1684.13	1946.94	1.0	0.00	0.00
phi[1]		0.53	0.19	0.22	0.82	1684.13	1946.94	1.0	0.00	0.00
r2		0.51	0.14	0.30	0.74	1592.56	1715.70	1.0	0.00	0.00
sigma		0.84	0.04	0.78	0.92	2975.67	2702.13	1.0	0.00	0.00
u_village[0]		0.51	0.32	0.06	1.06	882.40	839.53	1.0	0.01	0.01
u_village[1]		1.25	0.34	0.77	1.86	920.83	850.41	1.0	0.01	0.01
u_village[2]		0.21	0.32	-0.23	0.78	930.51	823.35	1.0	0.01	0.01
u_village[3]		-0.14	0.31	-0.57	0.41	892.52	871.48	1.0	0.01	0.01
u_village[4]		-0.17	0.32	-0.61	0.38	923.43	884.16	1.0	0.01	0.01

R

```

library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- import("jax.numpy")

set.seed(0)

```

```

N          <- 200L
N_villages <- 5L
village_id <- sample(0L:(N_villages - 1L), N, replace = TRUE)
x          <- rnorm(N)
u_true     <- rnorm(N_villages, 0, 0.6)
y          <- 1.2 + 0.8 * x + u_true[village_id + 1L] + rnorm(N, 0, 0.8)

m$data_on_model <- list(
  y          = jnp$array(as.numeric(y),          dtype = jnp$float32),
  x          = jnp$array(as.numeric(x),          dtype = jnp$float32),
  village_id = jnp$array(as.integer(village_id), dtype = jnp$int32),
  N_villages = N_villages
)

model <- function(y, x, village_id, N_villages) {
  sigma <- m$dist$exponential(1, name = "sigma")

  # R2D2: 2-component simplex (predictor + group variance)
  r2    <- m$dist$beta(1/3, 3, name = "r2")
  tau2  <- sigma^2 * r2 / (1 - r2)
  phi   <- m$dist$dirichlet(jnp$ones(2L), name = "phi")
  lam2  <- tau2 * phi

  alpha <- m$dist$normal(0, 0.5, name = "alpha")
  beta  <- m$dist$normal(0, jnp$sqrt(lam2[0L]), name = "beta")
  u_village <- m$dist$normal(0, jnp$sqrt(lam2[1L]),
                           shape = tuple(N_villages), name = "u_village")

  mu <- alpha + beta * x + u_village[village_id]
  m$dist$normal(mu, sigma, obs = y, name = "y_obs")
}

m$fit(model, progress_bar = FALSE)
m$summary()

```

i Note

In this two-component formulation the posterior of `phi[1]` quantifies how much of the total explained variance is attributable to village structure versus the fixed predictor. If the villages explain very little, `phi[1]` collapses toward zero and `sigma_village = sqrt(lam2[1])` is automatically regularized — without requiring a separate prior on the

group-level standard deviation.

When R2D2 Fails

Prior Misspecification

The Beta prior on R^2 is the most influential dial in the whole framework: it sets the upper ceiling on how much variance the coefficients can collectively explain. When this prior is misspecified — particularly too conservative relative to the truth — the posterior of every coefficient is pulled toward zero even when the data strongly contradict it. The effect is amplified in small samples, where the likelihood cannot fully overcome the prior.

The example below simulates a scenario with a genuinely strong effect (true $\beta = 2.5$, true $R^2 \approx 0.96$) and a small sample ($N = 20$). The conservative default `Beta(1/3, 3)` places its prior mass near $R^2 \approx 0.1$; an appropriately calibrated `Beta(3, 1)` places it near 0.75. The posterior mean of β differs substantially between the two priors — illustrating that R2D2's output is only as trustworthy as the domain knowledge encoded in the Beta.

Python

```
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

m = bf(platform="cpu")

np.random.seed(7)
N = 20
x = np.random.normal(0, 1, N).astype("float32")
y = (2.5 * x + np.random.normal(0, 0.5, N)).astype("float32")
# True  $R^2 = 6.25 / (6.25 + 0.25) = 0.96$ 

m.data_on_model = {
    "y": jnp.array(y),
    "x": jnp.array(x),
}

def model_conservative(y, x):
    """Beta(1/3, 3): prior mean  $R^2 = 0.10$  - badly misspecified here."""
    sigma = m.dist.exponential(1, name="sigma")
```

```

r2      = m.dist.beta(1/3, 3, name="r2")          # prior mean  0.10
tau2    = sigma**2 * r2 / (1 - r2)
alpha   = m.dist.normal(0, 0.5, name="alpha")
beta    = m.dist.normal(0, jnp.sqrt(tau2), name="beta")
m.dist.normal(alpha + beta * x, sigma, obs=y, name="y_obs")

def model_calibrated(y, x):
    """Beta(3, 1): prior mean R^2  0.75 - closer to truth."""
    sigma = m.dist.exponential(1, name="sigma")
    r2     = m.dist.beta(3, 1, name="r2")          # prior mean  0.75
    tau2   = sigma**2 * r2 / (1 - r2)
    alpha  = m.dist.normal(0, 0.5, name="alpha")
    beta   = m.dist.normal(0, jnp.sqrt(tau2), name="beta")
    m.dist.normal(alpha + beta * x, sigma, obs=y, name="y_obs")

print("=== Conservative prior (Beta(1/3, 3)) - true beta = 2.5 ===")
m.fit(model_conservative, progress_bar=False)
m.summary()

print("=== Calibrated prior (Beta(3, 1)) - true beta = 2.5 ===")
m.fit(model_calibrated, progress_bar=False)
m.summary()

```

```

jax.local_device_count 32
=== Conservative prior (Beta(1/3, 3)) - true beta = 2.5 ===
=== Calibrated prior (Beta(3, 1)) - true beta = 2.5 ===

```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
alpha	-0.01	0.14	-0.24	0.22	3544.50	2450.31	1.0	0.0	0.0
beta	2.82	0.17	2.55	3.09	3563.48	2664.57	1.0	0.0	0.0
r2	0.90	0.08	0.76	0.99	3130.15	2267.30	1.0	0.0	0.0
sigma	0.65	0.12	0.49	0.86	2958.33	2602.14	1.0	0.0	0.0

Warning

This is not a pathological edge case. Any dataset where the true model is highly predictive ($R^2 > 0.5$) and the sample is small risks systematic under-estimation under the typical `Beta(1/3, 3)` default. Always verify that the Beta hyperparameters are consistent with substantive domain knowledge — a prior predictive check on R^2 before fitting is strongly recommended.

Non-Normal Outcomes

The key identity $\tau^2 = \sigma^2 R^2 / (1 - R^2)$ requires σ^2 to be a free residual variance. For distributions with no dispersion parameter — Binomial, Poisson, Categorical — this parameter does not exist, and the derivation breaks. Applying R2D2 to such models requires either:

1. **Fixing σ to the canonical scale** (e.g., $\sigma = 1$ on the logit scale for Bernoulli), which destroys the R^2 -interpretation of the prior: τ^2 now scales with a constant, not with the actual residual noise.
2. **Marginalizing over the latent continuous variable** (e.g., in a probit model) to recover a well-defined R^2 , which considerably complicates the implementation.

In practice, applying R2D2 naively to a Binomial or Poisson model produces a prior whose shape depends on an arbitrary constant — the resulting shrinkage is no longer tied to any interpretable goodness-of-fit metric. For discrete outcomes, horseshoe or spike-and-slab priors are better-motivated alternatives.

Mathematical Details

Hierarchical Mechanism

The R2D2 prior can be expressed as the following hierarchical model, where K is the number of predictors:

$$R^2 \sim \text{Beta}(a, b)$$

$$\tau^2 = \frac{\sigma^2 R^2}{1 - R^2}$$

$$\phi \sim \text{Dirichlet}(1, \dots, 1)$$

$$\lambda_j^2 = \tau^2 \phi_j, \quad j = 1, \dots, K$$

$$\beta_j \sim \text{Normal}(0, \lambda_j^2), \quad j = 1, \dots, K$$

Where:

- R^2 is the expected coefficient of determination, i.e., the fraction of the outcome variance we expect the model to explain.

- σ is the standard deviation of the residual (noise) term.
- τ^2 is the global variance, i.e., the total explained variance available to the model.
- ϕ_j is the Dirichlet weight assigned to the j -th predictor, with $\sum_j \phi_j = 1$.
- λ_j^2 is the local variance of the j -th coefficient.
- β_j is the regression coefficient of the j -th predictor.

Extension to Varying Effects

To apply the R2D2 prior to varying effects, the fundamental logic remains the same, but the Dirichlet decomposition is expanded to include the variance components of the grouping factors. Instead of the global variance τ^2 being distributed solely among the fixed predictors, the simplex ϕ is broadened so that the model variance is partitioned into:

- the variance explained by population-level (fixed) effects, and
- the variance explained by group-level (varying) effects.

For example, a varying intercept with variance σ_{group}^2 is treated like any other local variance: it receives a slice of the global τ^2 through a Dirichlet weight. This forces fixed covariates and varying effects to actively compete for the total explained variance.

This approach is particularly useful for highly structured, non-independent data:

- *Dyadic varying effects*: when modeling social interaction networks, variance is typically partitioned into multi-membership structures such as sender, receiver, and dyadic effects. An R2D2 prior allocates variance across these distinct structural levels, regularizing them jointly rather than setting independent, often uninformative priors (e.g., Half-Normal or Exponential) on each variance component.
- *Spatial GLMMs*: when incorporating spatial autocorrelation terms, the Dirichlet decomposition balances the variance captured by the underlying spatial structure against that captured by local environmental covariates, providing adaptive shrinkage if the spatial signal is weak.

Definition of R^2

The R^2 that the Beta prior acts on is the model’s **explained-variance ratio** — signal variance over total variance:

$$R^2 = \frac{\text{Var}(\mathbf{x}^\top \beta)}{\text{Var}(\mathbf{x}^\top \beta) + \sigma^2} = \frac{\tau^2}{\tau^2 + \sigma^2},$$

which is exactly the identity $\tau^2 = \sigma^2 R^2 / (1 - R^2)$ rearranged. This is a property of the *data-generating process*, not of a particular sample — it is **not** the classical statistic $R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ reported in a regression table. The two agree in large samples under a well-specified model, but can differ substantially when N is small; a prior predictive check should always be read against the explained-variance definition above.

A second subtlety appears in mixed-effects models: one must distinguish between the *marginal* R^2 (variance explained by fixed effects alone) and the *conditional* R^2 (variance explained by fixed plus varying effects). To properly use R2D2 for varying effects, the Beta prior must target the conditional R^2 , so that the full signal-to-noise ratio is captured before the Dirichlet weights decompose it down the hierarchy.

Limitations

Warning

Normal outcomes only. The identity $\tau^2 = \sigma^2 R^2 / (1 - R^2)$ requires a free residual variance σ^2 . This exists in Gaussian linear models but not in Binomial, Poisson, or Categorical likelihoods. Applying R2D2 to discrete outcomes renders the Beta prior’s R^2 -interpretation meaningless (see *When R2D2 Fails* above).

Predictors must be standardized. The Dirichlet decomposition distributes *variance* across predictors. If predictors are on incomparable scales, $\lambda_j^2 = \tau^2 \phi_j$ conflates coefficient magnitude with predictor scale, and the simplex weights lose their “share of explained variance” meaning.

Symmetric competition may be insufficient for high sparsity. The flat $\text{Dirichlet}(1, \dots, 1)$ prior assigns equal prior weight to every predictor. When the true model is very sparse (K is large but only a handful of predictors matter), this produces ridge-like shrinkage — every coefficient is pulled toward zero equally — rather than the stronger, spike-at-zero regularization of a horseshoe or spike-and-slab prior. The latter are preferable when $p \gg n$ and sparsity is expected.

Prior sensitivity is amplified in small samples. The Beta hyperparameters (a, b) are directly influential when N is small or K is large relative to N . A prior predictive check — sampling R^2 and implied β values before fitting — is not optional; it is part of the model specification.

Cannot impose sign constraints on individual coefficients. The Dirichlet decomposition controls variance, not direction. If domain knowledge restricts a coefficient to be positive (or negative), that constraint must still be imposed separately (e.g., via a Half-Normal prior on that term or a monotone link function).

Regularizes coefficient magnitude, not model structure. R2D2 can shrink a coefficient toward zero but cannot remove a structural model component — a Gaussian process kernel, a varying-slope dimension, an entire likelihood family. It is a prior *within* a

fixed model architecture, not a tool for comparing architectures. See *R2D2 vs Structural Model Selection* below.

How to Use It Well

Tip

- **Run a prior predictive check on R^2 before fitting.** Sample your $\text{Beta}(a, b)$ and inspect the 5th–95th percentile against what you plausibly expect the model to explain. This is the single most effective safeguard against a misspecified prior.
- **Small sample with a suspected strong fit: do not use the $\text{Beta}(1/3, 3)$ default.** Its mass sits near $R^2 \approx 0.1$; against a process that truly explains most of the variance it inflates σ and shrinks the real coefficients. Widen it ($\text{Beta}(1, 1)$) or shift it up.
- **When in doubt, be less confident, not more.** A vague, mildly-wrong prior degrades gracefully; a confident, wrong prior — e.g. $\text{Beta}(5, 1)$ when the truth is a poor fit — produces the largest errors, over-stating R^2 and under-stating σ .
- **Low- R^2 problems (roughly 0.05–0.50, typical in ecology and quantitative biology): drop the Dirichlet concentration to $a_0 \approx 0.1$ –0.5.** The flat Dirichlet(1, ..., 1) is a ridge; $a_0 < 1$ gives horseshoe-like tails that keep the credible intervals honest and shrink the null coefficients harder. In tests at true $R^2 \approx 0.3$ with N large enough, $a_0 = 0.1$ raised interval coverage of the real effects from 1/5 to 4/5 at no sampler cost; below $a_0 \approx 0.1$ the simplex funnels and divergences return, so do not go lower without reparameterizing. When N is small relative to K , no value of a_0 rescues the fit — the problem is information, not the prior.
- **Match the $\text{Beta}(a, b)$ to your field, not to a default.** $\text{Beta}(1/3, 3)$ has infinite density at $R^2 = 0$; if the field reliably explains 10–40% of the variance, a prior *concentrated* in that band ($\text{Beta}(2, 6)$ or $\text{Beta}(1, 3)$, prior mean ≈ 0.25) is more honest, or fit (a, b) to the R^2 values reported in comparable studies (empirical Bayes).
- **Standardize every predictor — and consider standardizing y .** The Dirichlet decomposition distributes *variance*; on incomparable predictor scales $\lambda_j^2 = \tau^2 \phi_j$ conflates coefficient magnitude with predictor units and the simplex weights stop meaning “share of explained variance”. Scaling y additionally makes the $\sigma \sim \text{Exp}(1)$ prior scale-appropriate (then $\sigma \approx \sqrt{1 - R^2}$), which matters most at low R^2 where the residual dominates τ^2 .

- **Use a non-centered parameterization for the coefficients** — $\beta_j = z_j \lambda_j$ with $z_j \sim \text{Normal}(0, 1)$. It is mathematically identical to $\beta_j \sim \text{Normal}(0, \lambda_j^2)$ but avoids the funnel geometry that produces divergences and inflated $\hat{R}$ when $K > N$ and the prior admits $R^2 \rightarrow 1$.
- **Report divergences and $\hat{R}$.** This prior can look healthy in a summary table while the sampler is stressed underneath; the $R^2 \rightarrow 1$ region is stiff.

A worked simulation study of these effects — parameter recovery against a known truth at $N = 50$ and $N = 300$, how the estimates move as the residual noise (and hence the true R^2) is swept from ≈ 0.98 down to ≈ 0.43 , and the effect of the concentration a_0 in the low- R^2 regime — is in `BF/Test/R2D2/`.

Notes

Note

The main advantage of the R2D2 prior over independent coefficient variances is that it turns prior specification into a question about model-level expectations rather than about individual abstract parameters: how much of the outcome variance should this model explain, and how should that explained variance be shared among the predictors? Because the simplex constrains the weights to sum to one, the prior is *self-balancing* — a predictor that is not useful in explaining the outcome is automatically assigned a small share of the variance and shrunk towards zero, while the heavy tails of the induced coefficient distribution protect the estimation of genuinely large effects.

R2D2 vs Structural Model Selection

Note

R2D2 and PSIS-LOO model comparison (e.g., via `BayesForge.Diagnostic.compare`) answer *different questions* and are not interchangeable.

What R2D2 does: within a single, fixed model architecture it regularizes regression coefficients so that unnecessary predictors are shrunk toward zero. The 10-model spatial pipeline in this project tests whether *anthropogenic covariates* (disturbance frequency, local/global urbanization) are informative. R2D2 could in principle fold all 10 into one model and let the prior do the selection — but only for the fixed-effect predictors, and only for Normal outcomes.

What structural model selection does: it compares models that differ in *architecture* — presence or absence of a spatial GP kernel, 3D vs 4D individual-level MVN, Normal